Generación de imágenes cartoon con MPI y OpenMP

Práctico Final Integrador de Computación Avanzada · UADER · 2026

Resultados medidos el 28 de octubre de 2026 en una computadora Windows

1 Introducción y objetivos

El objetivo fue construir una imagen cartoon a partir de una imagen RGB y comparar una implementación secuencial con tres formas de paralelismo: MPI, OpenMP y una combinación de ambas. El análisis utiliza exclusivamente las ejecuciones conservadas en los CSV del proyecto. Cada salida paralela se comparó con la secuencial para el mismo tamaño, filtro y cantidad de niveles; la equivalencia binaria se confirmó en los doce casos exigidos por la consigna (UADER, 2026).

2 Fundamentos del procesamiento de imágenes

Una imagen RGB de 8 bits almacena tres canales por píxel, cada uno entre 0 y 255. Para detectar contornos se calcula una intensidad gris como 0,30R + 0,59G + 0,11B. El promedio espacial de una vecindad atenúa variaciones locales antes de aplicar Sobel, que estima cambios horizontales y verticales. La posterización reduce el número de valores posibles por canal y simplifica las regiones de color (Guerrero, 2017).

3 Algoritmo cartoon

El programa convierte la imagen a gris, aplica un promedio uniforme de 3 × 3 o 5 × 5 píxeles, calcula |Sx| + |Sy| con Sobel y marca un borde cuando el resultado supera el umbral 150. En paralelo lógico, cada canal RGB se cuantiza en 3 o 9 franjas, usando el punto medio de cada franja. La salida suma la máscara de bordes a la imagen posterizada y limita cada canal a 255. La opción de bordes oscuros resta esa máscara y limita inferiormente a cero; los benchmarks emplearon bordes claros.

4 Implementación secuencial

La referencia recorre todos los píxeles en un proceso. Las etapas producen vectores separados de gris, imagen borrosa y bordes; luego se posteriza una copia RGB y se combinan los resultados. El cronómetro steady_clock comienza después de leer el PPM y termina antes de escribir la salida. Esta medición define la base para cada speedup.

5 Implementación MPI

El proceso raíz lee el PPM y difunde ancho y alto con MPI_Bcast. Las filas se reparten en bloques contiguos, asignando las filas sobrantes a los primeros procesos. MPI_Scatterv envía bloques de tamaño variable con halos, y MPI_Gatherv reconstruye la imagen con las filas propias de cada proceso. La medición con MPI_Wtime comienza antes del reparto y termina después de la recolección, por lo que incluye esas comunicaciones. El código informa error si la lectura, la configuración o la escritura falla.

6 Implementación OpenMP

Las etapas de gris, borroneo, Sobel, posterización y combinación emplean parallel for. Cada iteración escribe una posición distinta; los acumuladores del promedio son variables locales del bucle. La barrera implícita al finalizar cada región asegura que la etapa siguiente lea una matriz completa. Se deshabilitó el ajuste dinámico de hilos y se informó el número efectivo. Se comprobaron 1, 2, 4 y 8 hilos.

7 Implementación híbrida MPI y OpenMP

Cada proceso MPI recibe un bloque con halos y ejecuta las etapas de píxeles con hilos OpenMP. Las llamadas MPI quedan fuera de las regiones paralelas. Se midieron 2 procesos × 2 hilos, 2 × 4 y 4 × 2. El programa informa el mínimo y máximo de hilos efectivos observados entre procesos; la prueba de casos límite verificó que coincidieran con la configuración solicitada.

8 División del problema

El borroneo de tamaño k necesita k/2 filas vecinas a cada lado, mientras que Sobel necesita una fila más sobre el resultado borroso. Por ello se distribuye un halo de k/2 + 1 filas: dos para 3 × 3 y tres para 5 × 5. Solo las filas propias vuelven al proceso raíz. El reparto contiguo simplifica las transferencias, aunque agrega cálculo redundante en los halos y copias de memoria.

9 Comunicación y sincronización

MPI_Barrier alinea el inicio de la región cronometrada y las operaciones colectivas coordinan reparto y recolección. No hay intercambio entre procesos durante el cálculo porque cada uno recibió el halo completo. Las barreras implícitas de OpenMP separan etapas dependientes. La comparación byte por byte de todas las salidas frente a la referencia ayuda a detectar errores en fronteras de bloques y carreras visibles en el resultado.

10 Metodología y equipo de pruebas

Se generaron escenas RGB deterministas de 800 × 800, 2000 × 2000 y 5000 × 5000 píxeles. Para cada tamaño se probaron filtros de 3 × 3 y 5 × 5 y posterización de 3 y 9 niveles: doce casos. Cada caso se ejecutó tres veces en once configuraciones, para un total de 396 mediciones. Los tiempos son los impresos por cada programa y excluyen lectura y escritura. El equipo utilizado tiene un AMD Ryzen 7 5700 y 16 procesadores lógicos; ejecutó Windows build 26200, GCC 16.2 y Microsoft MPI 10.1.3. Las pruebas se hicieron en un solo equipo, de modo que MPI mide comunicación entre procesos locales. La auditoría de CSV verificó tres repeticiones únicas por configuración, tiempos positivos, promedios, fórmulas y un único SHA-256 de salida por cada uno de los doce casos.

Para el análisis se construyeron siete gráficos a partir de resumen.csv (promedios, speedup y eficiencia) y de mediciones.csv (variabilidad de las tres repeticiones). Speedup = tiempo secuencial / tiempo paralelo de la misma carga; eficiencia = speedup / unidades de ejecución (hilos en OpenMP, procesos en MPI, procesos × hilos en la híbrida). Cuando un gráfico promedia varias cargas, se indica qué cargas incluye. Las causas propuestas para los comportamientos observados se presentan como hipótesis, porque los datos no las miden.

11 Resultados de rendimiento

OpenMP con 8 hilos obtuvo el menor promedio en 8 de los doce casos; la configuración híbrida 4 × 2 lo obtuvo en 4. La tabla 1 muestra la configuración ganadora de cada caso. El archivo resumen.csv conserva las 132 filas con procesos, hilos, tiempo, speedup y eficiencia; mediciones.csv conserva las 396 observaciones individuales.

Imagen	Filtro	Niv.	Versión	Proc.	Hilos	Tiempo s	Speedup	Efic.
800x800	3×3	3	OpenMP	1	8	0.0057	1.86	0.23
800x800	3×3	9	Híbrida	4	2	0.0044	2.50	0.31
800x800	5×5	3	Híbrida	4	2	0.0053	2.89	0.36
800x800	5×5	9	OpenMP	1	8	0.0053	2.96	0.37
2000x2000	3×3	3	OpenMP	1	8	0.0243	2.56	0.32
2000x2000	3×3	9	OpenMP	1	8	0.0257	2.71	0.34
2000x2000	5×5	3	OpenMP	1	8	0.0330	3.27	0.41
2000x2000	5×5	9	Híbrida	4	2	0.0306	3.31	0.41
5000x5000	3×3	3	OpenMP	1	8	0.1207	3.35	0.42
5000x5000	3×3	9	OpenMP	1	8	0.1227	3.26	0.41
5000x5000	5×5	3	OpenMP	1	8	0.1710	3.65	0.46
5000x5000	5×5	9	Híbrida	4	2	0.1729	3.85	0.48

Tabla 1. Configuración con menor tiempo promedio en cada caso. Speedup y eficiencia corresponden a esa misma configuración.

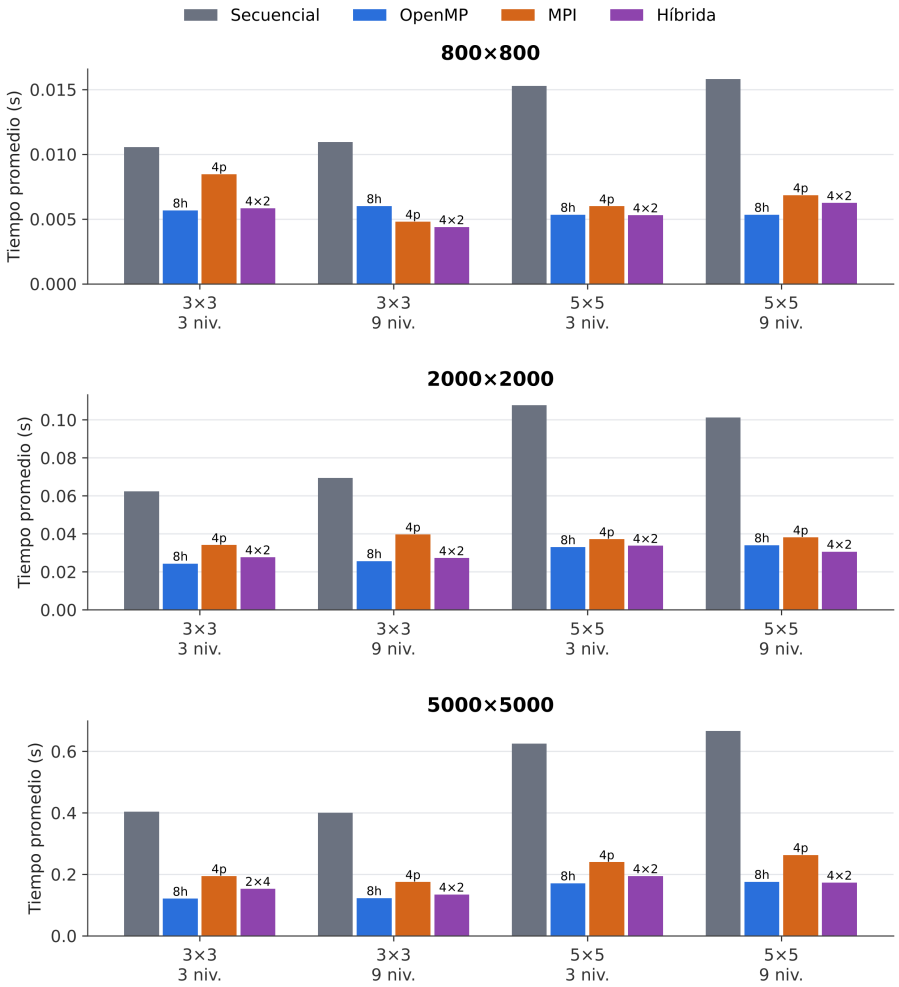


Figura 1. Tiempo promedio de la mejor configuración de cada versión en las doce cargas. Etiqueta sobre cada barra: h = hilos, p = procesos, p×h = procesos × hilos. Cada panel usa su propia escala vertical porque los tamaños difieren en órdenes de magnitud; no son comparables entre paneles.

Qué muestra. para cada carga (tamaño, filtro y niveles), el menor tiempo promedio de tres repeticiones alcanzado por OpenMP, MPI y la híbrida, junto con el tiempo secuencial de referencia.

Mejor rendimiento. OpenMP con 8 hilos fue la configuración más rápida en 8 de las 12 cargas y la híbrida 4 × 2 en 4; las cargas ganadas por la híbrida tienen todas filtro 5 × 5 o 9 niveles. MPI con 4 procesos no fue la más rápida en ninguna (0 de 12).

Comportamiento. En las 12 cargas la mejor configuración de cada versión paralela fue más rápida que la secuencial. El speedup de la configuración ganadora varía entre 1,86 y 3,85.

Conclusión defendible. OpenMP con 8 hilos lideró la mayoría de los casos, pero su ventaja sobre la híbrida 4 × 2 no es sistemática y en imágenes pequeñas cae dentro de la dispersión de las mediciones (Figura 7).

El filtro de 5 × 5 exige hasta 25 contribuciones por píxel, frente a 9 del filtro 3 × 3. En la versión secuencial, la relación media tiempo 5 × 5 / tiempo 3 × 3, promediada sobre los dos niveles, fue 1,45 en 800 × 800, 1,59 en 2000 × 2000 y 1,61 en 5000 × 5000. En algunas configuraciones paralelas pequeñas la relación cambia por variación de ejecución y sobrecarga, por lo que el costo del filtro no debe inferirse de un solo punto.

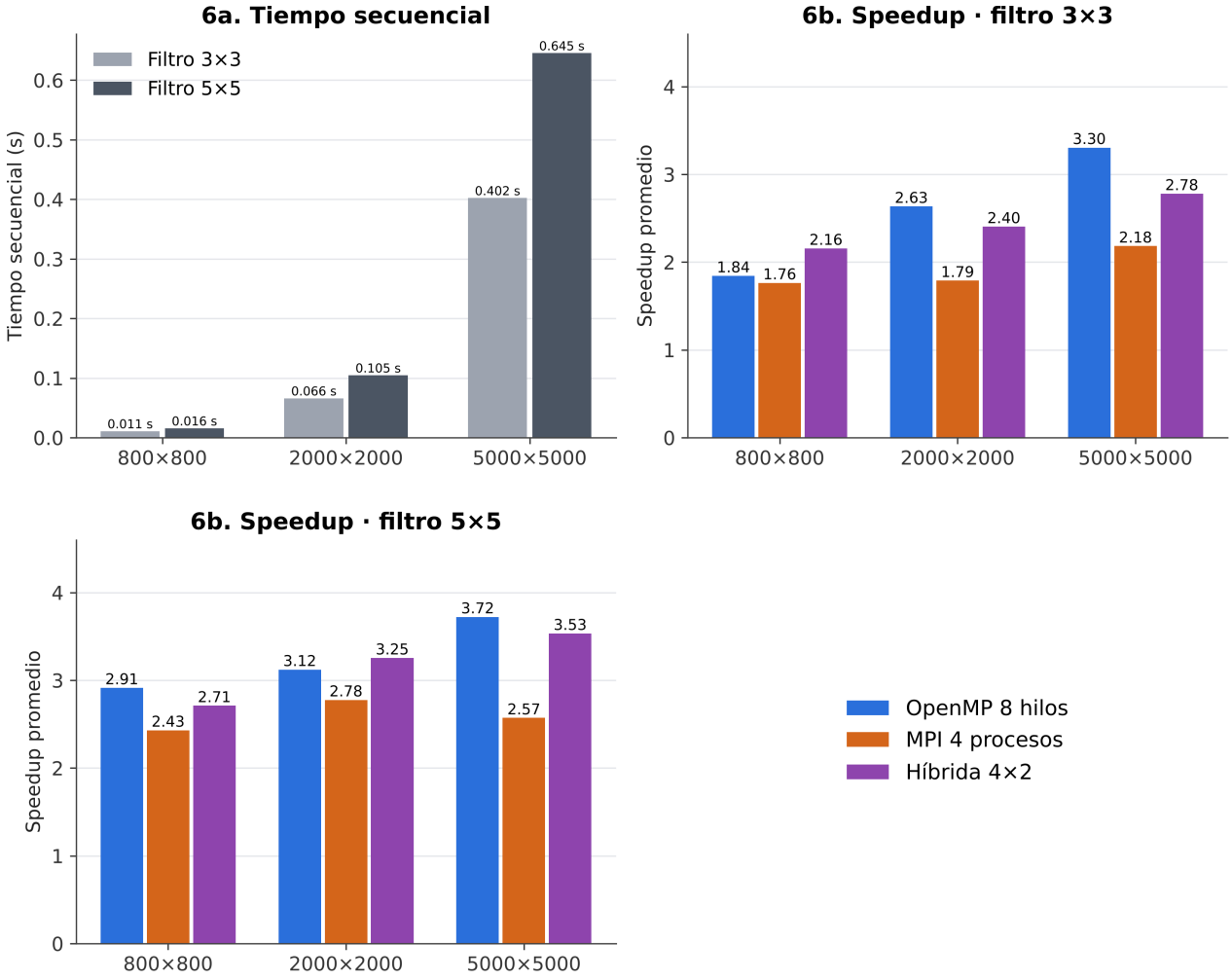


Figura 2. Influencia del filtro y del tamaño de imagen. (a) Tiempo secuencial promedio por tamaño y filtro, con los valores sobre las barras. (b) Speedup promedio de OpenMP con 8 hilos, MPI con 4 procesos e híbrida 4 × 2 por tamaño, un panel por filtro; promedio de 3 y 9 niveles. Las unidades de ejecución difieren (8, 4 y 8): se compara rendimiento, no eficiencia.

Qué muestra. cómo cambia el tiempo secuencial al pasar de 3 × 3 a 5 × 5 y de un tamaño de imagen a otro, y si una carga mayor permite aprovechar mejor el paralelismo.

Mejor rendimiento. El mayor speedup promedio de OpenMP con 8 hilos se obtuvo con filtro 5 × 5 en 5000 × 5000 (3,72). Con filtro 5 × 5 en 2000 × 2000 la híbrida 4 × 2 (3,25) superó a OpenMP con 8 hilos (3,12).

Comportamiento. El tiempo secuencial crece con la cantidad de píxeles y con el tamaño del filtro. El speedup promedio de OpenMP con 8 hilos sube de 2,59 con filtro 3 × 3 a 3,25 con 5 × 5, y también crece con el tamaño de la imagen.

Conclusión defendible. una carga computacional mayor (filtro 5 × 5 o imágenes más grandes) aprovecha mejor el paralelismo, porque los costos fijos pesan menos frente al cálculo. Es una asociación observada en estas doce cargas; el código no mide por separado cada costo fijo.

12 Speedup y eficiencia

Para cada caso se calculó speedup = promedio secuencial / promedio paralelo de la misma imagen, filtro y niveles. La eficiencia divide ese speedup por hilos para OpenMP, procesos para MPI o procesos × hilos para la híbrida. La tabla 2 resume la media de los cuatro speedups de filtro y niveles para cada tamaño. Son medias descriptivas de razones por caso, no el cociente entre tiempos agregados.

Imagen	OpenMP 8 h	MPI 4 p	Híbrida 2×4	Híbrida 4×2
800×800	2,38	2,10	2,17	2,43
2000×2000	2,88	2,28	2,52	2,83
5000×5000	3,51	2,38	2,94	3,16

Tabla 2. Speedup medio (veces) sobre las cuatro combinaciones de filtro y niveles de cada tamaño.

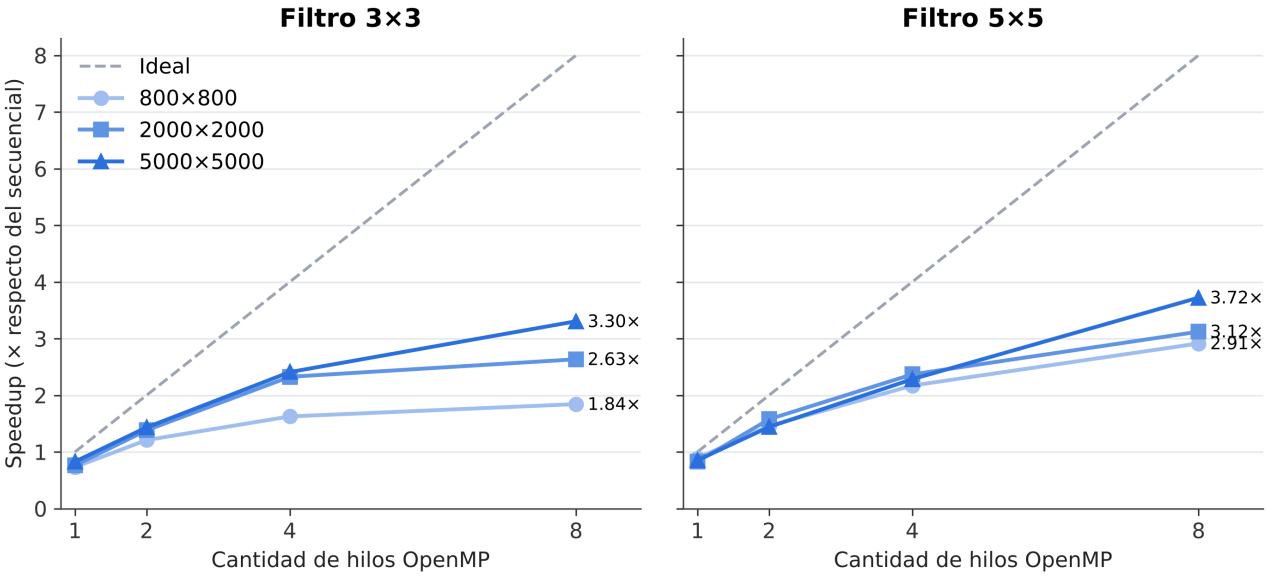


Figura 3. Speedup de OpenMP con 1, 2, 4 y 8 hilos. Cada línea es un tamaño de imagen (promedio de 3 y 9 niveles); un panel por filtro. La línea punteada es el speedup ideal (igual a la cantidad de hilos).

Qué muestra. cuánto mejora el tiempo al aumentar los hilos, comparado con la mejora ideal, para cada tamaño y filtro.

Mejor rendimiento. 8 hilos, con speedup promedio 2,92 (eficiencia 0,37); el valor más alto fue 3,72 en 5000 × 5000 con filtro 5 × 5.

Comportamiento. Con 1 hilo OpenMP es más lento que la referencia secuencial (speedup 0,82). El promedio sube a 1,42 con 2 hilos, 2,20 con 4 y 2,92 con 8: cada duplicación suma menos que la anterior, y la eficiencia baja de 0,71 a 0,55 y 0,37. Con 8 hilos el speedup crece con el tamaño: 2,38 (800 × 800), 2,88 (2000 × 2000) y 3,51 (5000 × 5000).

Conclusión defendible. OpenMP escala con rendimientos marginales decrecientes y escala mejor con más trabajo por imagen. El Ryzen 7 5700 tiene 8 núcleos físicos y 16 hilos lógicos, de modo que 8 hilos no superan los núcleos físicos; por lo tanto la caída de eficiencia no puede atribuirse solo a falta de núcleos. Causas posibles, no medidas aquí: ancho de banda de memoria compartido, costo de iniciar regiones paralelas y actividad del sistema en un equipo compartido.

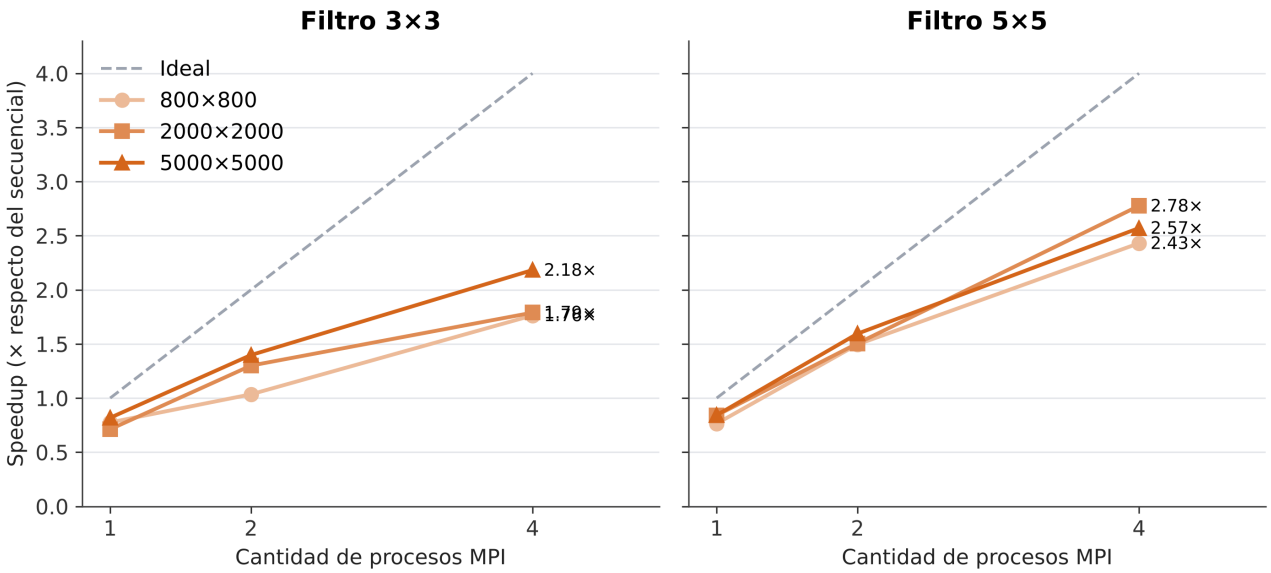


Figura 4. Speedup de MPI con 1, 2 y 4 procesos. Cada línea es un tamaño de imagen (promedio de 3 y 9 niveles); un panel por filtro. Línea punteada: speedup ideal. El tiempo de MPI incluye distribución y recolección de datos; no hay una medición independiente del costo de comunicación.

Qué muestra. el speedup en función de la cantidad de procesos, para imágenes pequeñas y grandes, contra el comportamiento ideal.

Mejor rendimiento. 4 procesos, con speedup promedio 2,25; ninguna carga alcanzó el ideal de 4.

Comportamiento. Con 1 proceso MPI es más lento que la referencia (speedup 0,79); con 2 procesos llega a 1,39 y con 4 a 2,25. Con 4 procesos el speedup pasa de 2,10 en 800 × 800 a 2,38 en 5000 × 5000: la mejora con el tamaño existe pero es moderada, menor que la de OpenMP.

Conclusión defendible. con la misma cantidad de unidades MPI y OpenMP rinden parecido (4 unidades: 2,25 contra 2,20), pero OpenMP llegó a 8 hilos y MPI solo se evaluó hasta 4 procesos. La diferencia entre MPI con 1 proceso y la referencia es compatible con un costo de distribución y recolección, pero estos datos no lo aíslan de otras diferencias de implementación.

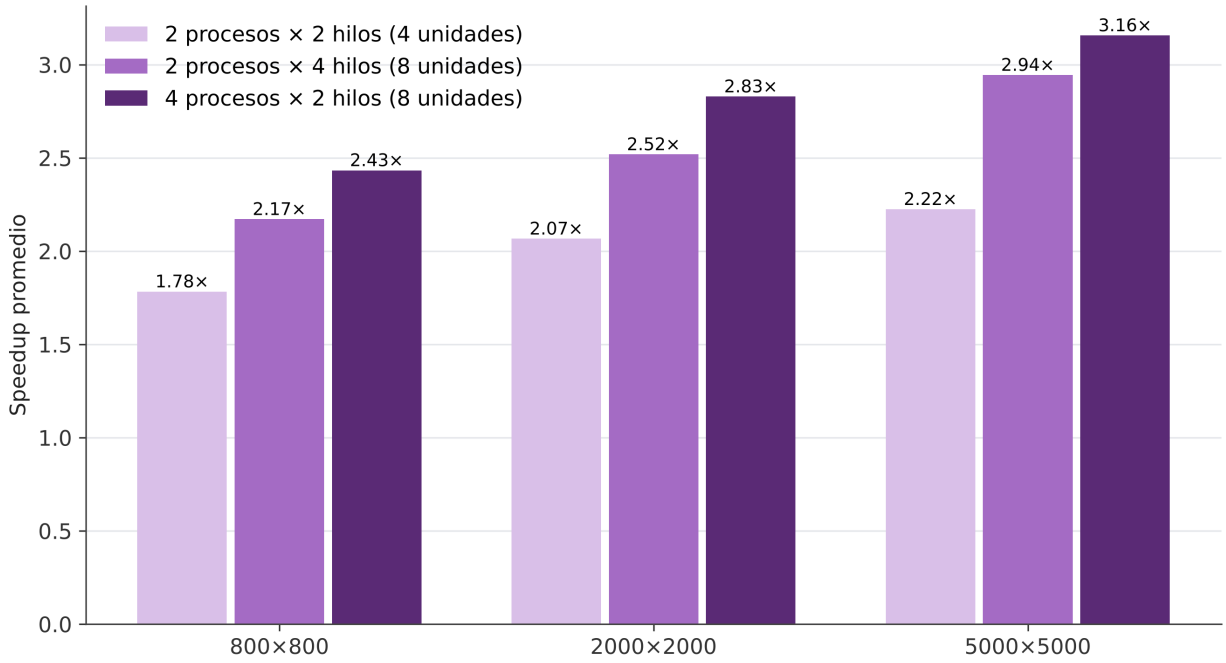


Figura 5. Speedup promedio de la versión híbrida por tamaño de imagen, para 2 procesos × 2 hilos, 2 × 4 y 4 × 2 (promedio de las cuatro combinaciones de filtro y niveles). 2 × 4 y 4 × 2 usan las mismas ocho unidades de ejecución.

Qué muestra. el rendimiento de tres formas de repartir las unidades entre procesos MPI e hilos OpenMP, y en particular si 2 × 4 y 4 × 2, con ocho unidades cada una, rinden distinto.

Mejor rendimiento. La híbrida 4 × 2, con speedup promedio 2,81, frente a 2,54 de la 2 × 4 y 2,02 de la 2 × 2.

Comportamiento. Con las mismas ocho unidades, 4 × 2 fue más rápida que 2 × 4 en 11 de las 12 cargas; en promedio, 2 × 4 tardó 12 % más. La diferencia aparece en los tres tamaños.

Conclusión defendible. usar más unidades no basta: repartir entre más procesos y menos hilos (4 × 2) rindió mejor que lo contrario en este equipo. Aun así, la híbrida 4 × 2 no superó sistemáticamente a OpenMP con 8 hilos (Figura 1). Una causa posible, no medida, es que los procesos MPI trabajan con memoria separada mientras que los hilos comparten caché y ancho de banda.

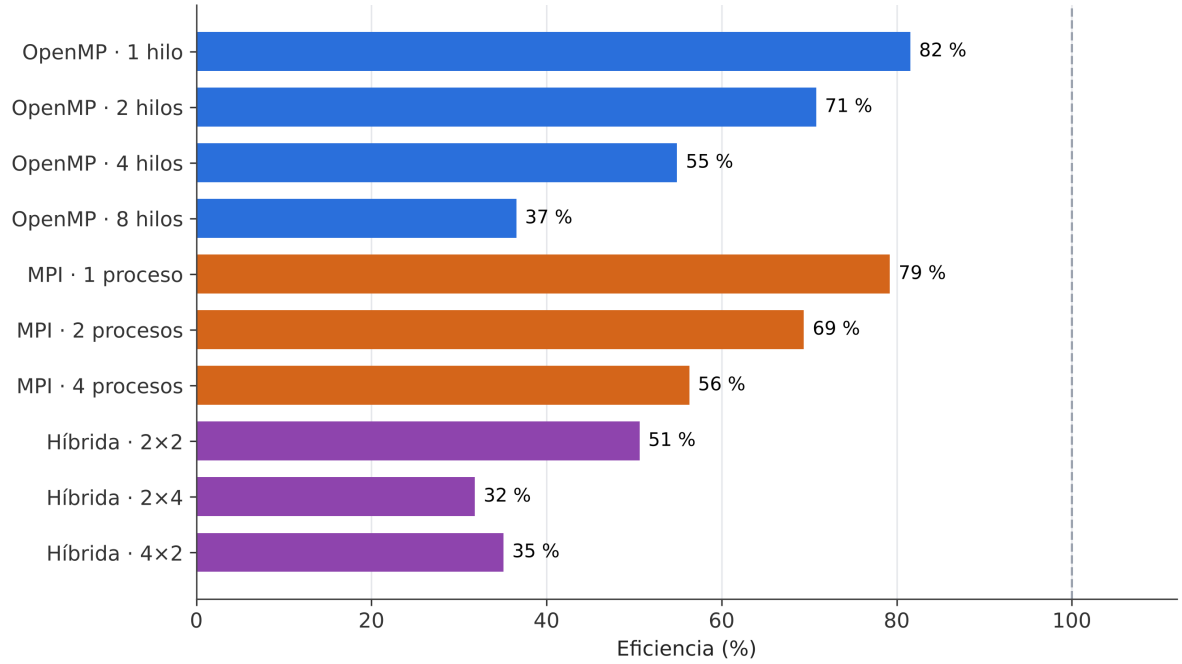


Figura 6. Eficiencia de las configuraciones paralelas, en porcentaje, promediada sobre las doce cargas. Eficiencia = speedup / unidades de ejecución: hilos en OpenMP, procesos en MPI y procesos × hilos en la híbrida (4, 8 y 8 unidades). La línea punteada marca el ideal (100 %).

Qué muestra. qué fracción del rendimiento ideal aprovecha cada configuración según sus unidades de ejecución.

Mejor rendimiento. Entre las configuraciones con más de una unidad, las de 2 unidades: OpenMP con 2 hilos (71 %) y MPI con 2 procesos (69 %).

Comportamiento. La eficiencia cae al sumar unidades. Con 4 unidades: MPI 56 %, OpenMP 55 % e híbrida 2 × 2 51 %. Con 8: OpenMP 37 %, híbrida 4 × 2 35 % e híbrida 2 × 4 32 %, la menos eficiente de todas.

Conclusión defendible. hay un compromiso entre velocidad absoluta y eficiencia: las configuraciones más rápidas, de 8 unidades, son también las menos eficientes, y ninguna alcanza el 100 %. Las configuraciones de una unidad quedan por debajo del 100 %, lo que es compatible con un costo fijo de la versión paralela (no medido por separado).

En 5000 × 5000, el speedup medio de OpenMP con 8 hilos fue 3,51 y el de MPI con 4 procesos fue 2,38. Las dos configuraciones híbridas de ocho unidades obtuvieron 2,94 (2 × 4) y 3,16 (4 × 2). La eficiencia correspondiente a OpenMP de 8 hilos fue aproximadamente 0,44, frente a 0,39 para la híbrida 4 × 2, al promediar los cuatro casos. El aumento de unidades mejora el tiempo, pero con rendimiento marginal decreciente.

13 Variabilidad de las mediciones

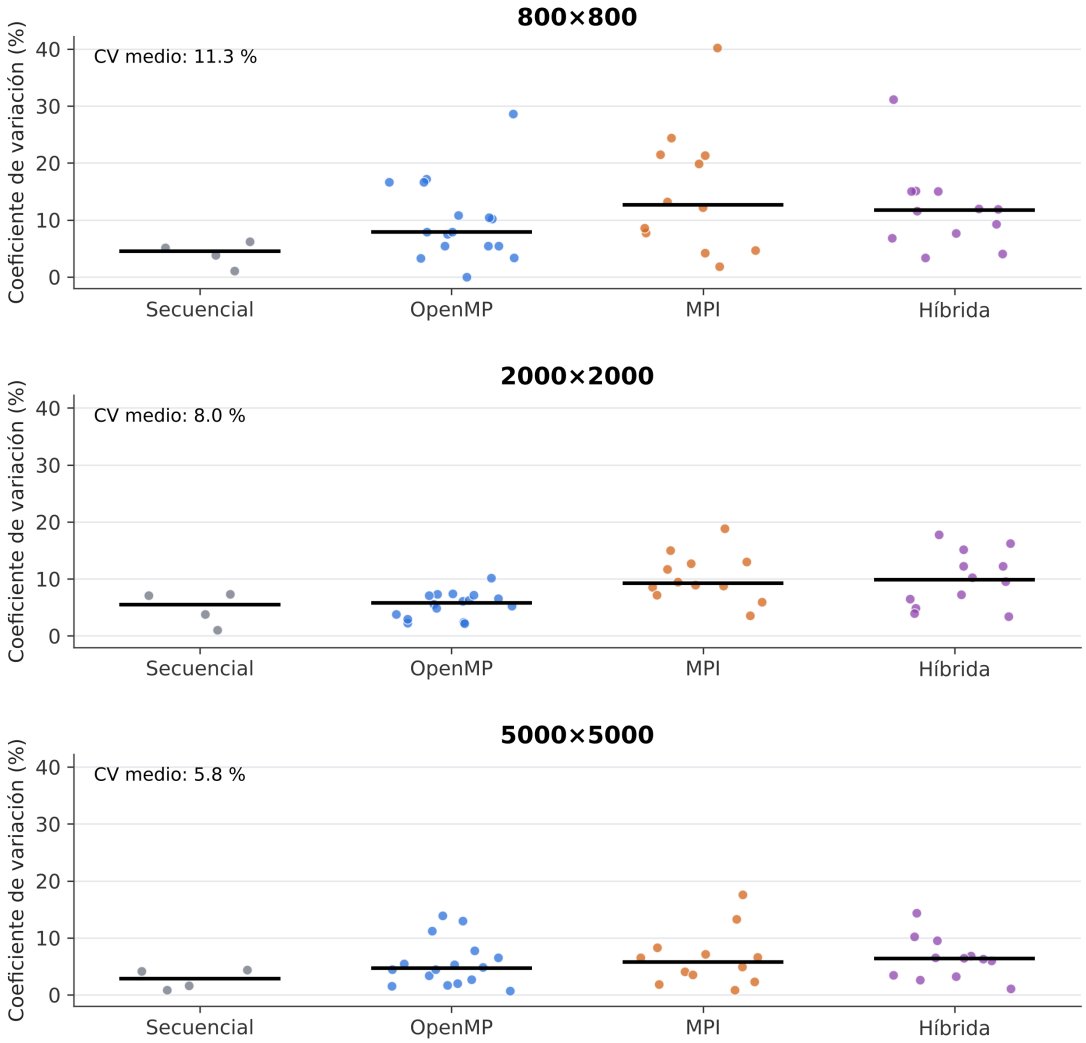


Figura 7. Variabilidad de las tres repeticiones. Cada punto es una configuración de una carga; su valor es el coeficiente de variación (desvío estándar muestral / media, en %). La raya negra es la mediana de cada versión. Un panel por tamaño de imagen.

Qué muestra. qué tan confiable es cada tiempo promedio: cuánto difieren entre sí las tres repeticiones de una misma configuración.

Comportamiento. El coeficiente de variación medio es 11,3 % en 800 × 800, 8,0 % en 2000 × 2000 y 5,8 % en 5000 × 5000; el máximo individual es 40 %. Las imágenes chicas son las más ruidosas; es probable que tiempos de pocos milisegundos se afecten más por la actividad del sistema (hipótesis, no medida).

Conclusión defendible. los speedups de 800 × 800 deben leerse con cautela, y las diferencias pequeñas entre configuraciones (por ejemplo OpenMP 8 hilos contra híbrida 4 × 2) caen dentro de la variabilidad. En contraste, la corrección no depende del ruido de tiempo: para cada una de las 12 cargas todas las configuraciones produjeron el mismo SHA-256 de salida (12 de 12 cargas).

14 Comparación de implementaciones

OpenMP fue la opción más rápida en la mayoría de los casos medidos en esta máquina compartida. MPI escaló al pasar de uno a cuatro procesos, aunque su ejecución de un solo proceso fue más lenta que la referencia; además del cómputo, su tiempo incluye Scatterv, Gatherv y las copias de bloques. Este resultado es compatible con un costo de comunicación, pero el ensayo no separa ese costo de las otras diferencias de implementación. La híbrida 4 × 2 ganó cuatro casos y superó a la 2 × 4 en la media de speedup de 5000 × 5000. No se extrapolan estas comparaciones a un clúster ni a otros procesadores.

El promedio de speedup de OpenMP con 8 hilos creció de 2,38 en 800 × 800 a 2,88 en 2000 × 2000 y 3,51 en 5000 × 5000. El de MPI con 4 procesos pasó de 2,10 a 2,28 y 2,38. El aumento del trabajo por ejecución amortiza una parte de los costos fijos de iniciar regiones paralelas y mover datos, aunque el código no mide cada componente por separado. Con un único hilo, OpenMP alcanzó speedups medios de 0,81, 0,80 y 0,84 según el tamaño; con un solo proceso, MPI obtuvo 0,77, 0,78 y 0,83. Esas variantes ejecutan trabajo adicional y por ello no son equivalentes en costo a la referencia secuencial.

La distribución MPI de este programa se hace desde el proceso raíz mediante Scatterv de bloques ya ampliados; no se realiza un intercambio directo de halos entre vecinos durante el cálculo. Esta decisión simplifica el control de fronteras, pero agrega bytes enviados y trabajo redundante a medida que crece el número de procesos. La alternativa híbrida reduce el tamaño del bloque por proceso y lo divide entre hilos. En los cuatro casos de 5000 × 5000, la media de speedup fue 2,94 para 2 × 4 y 3,16 para 4 × 2; la diferencia describe este ensayo y no demuestra que 4 × 2 sea siempre superior.

15 Ventajas desventajas y limitaciones

La secuencial ofrece una referencia simple. OpenMP evita repartir la imagen entre espacios de memoria y requiere cuidar la independencia de las escrituras y las barreras entre etapas. MPI permite pensar el algoritmo para memoria distribuida, a cambio de halos y comunicaciones. La híbrida combina ambos esquemas, pero agrega sincronización y copias; una mayor cantidad de unidades no garantiza eficiencia alta. Se usó PPM P6 para evitar dependencias de decodificación en C++, con mayor tamaño de archivo que JPEG o PNG. Las entradas de benchmark son sintéticas y deterministas; una fotografía real de 1330 × 1002 se procesó aparte y sus cuatro salidas coincidieron en SHA-256. Las mediciones de tiempos cortos pueden fluctuar por la resolución del reloj y la actividad del sistema; hay tres repeticiones, sin intervalos de confianza.

El alcance experimental está limitado a un equipo Windows y a una implementación concreta de cada modelo. La medida secuencial y OpenMP abarca solo cómputo, mientras que MPI e híbrida incluyen distribución y reunión de datos; por ello el speedup compara el tiempo observado de cada programa, no kernels de cálculo idénticos. No se separaron tiempos de comunicación, creación de hilos, asignación de memoria ni entrada y salida. La repetición de tres corridas permite calcular medias y observar dispersión individual en mediciones.csv (Figura 7), pero no permite afirmar diferencias pequeñas con significación estadística.

16 Conclusiones

Las cuatro implementaciones compilaron y produjeron resultados equivalentes en toda la matriz exigida. Las pruebas adicionales compararon 260 salidas pequeñas píxel por píxel con una referencia independiente y comprobaron 32 rechazos de entradas, parámetros o destinos inválidos. En el hardware utilizado, OpenMP con 8 hilos lideró ocho casos y la híbrida 4 × 2 lideró cuatro. El rendimiento mejoró con más trabajo por imagen, pero la eficiencia permaneció por debajo de uno al aumentar las unidades. La distribución por filas con halos resolvió correctamente las dependencias espaciales sin artefactos en las fronteras de proceso.

Del análisis de los gráficos se desprende además que: (i) el speedup promedio máximo fue 2,92 con OpenMP de 8 hilos, con eficiencia de 0,37; (ii) con una sola unidad, OpenMP y MPI son más lentas que la referencia; (iii) con las mismas ocho unidades, la híbrida 4 × 2 superó a la 2 × 4 en 11 de 12 cargas; (iv) filtros mayores e imágenes más grandes aprovechan mejor el paralelismo; y (v) la variabilidad es alta en 800 × 800 y baja en 5000 × 5000, por lo que las conclusiones más firmes provienen de las imágenes grandes.

17 Resumen para la defensa oral

- **Corrección primero.** Las once configuraciones producen exactamente la misma imagen (mismo SHA-256) en las 12 cargas de la matriz; los tiempos se comparan entre resultados idénticos.
- **El paralelismo ayuda, pero no es ideal.** La mejor aceleración promedio fue 2,92× con OpenMP de 8 hilos, con eficiencia de 37 %. Cada duplicación de hilos o procesos suma menos que la anterior.
- **El costo fijo existe.** Con una sola unidad, OpenMP (0,82) y MPI (0,79) son más lentas que la secuencial. Para MPI el tiempo incluye distribución y recolección, pero no se midió la comunicación por separado.
- **OpenMP contra MPI.** Con 4 unidades rinden parecido (2,20 contra 2,25). OpenMP llegó a 8 hilos; MPI solo se probó hasta 4 procesos.
- **Híbrida.** Con las mismas 8 unidades, 4 × 2 fue mejor que 2 × 4 en 11 de 12 cargas: no alcanza con usar más recursos, importa cómo se reparten. Aun así, OpenMP con 8 hilos ganó en 8 cargas y la híbrida 4 × 2 en 4.
- **La carga importa.** El speedup de OpenMP con 8 hilos crece de 2,38 a 3,51 al pasar de 800 × 800 a 5000 × 5000, y es mayor con filtro 5 × 5 (3,25) que con 3 × 3 (2,59).
- **Límites honestos.** En 800 × 800 la variabilidad es alta (coeficiente de variación medio 11,3 %); un solo equipo, tres repeticiones, sin medición aislada de comunicación. Las causas propuestas son hipótesis y no se extrapolan a un clúster.

18 Referencias bibliográficas

Guerrero, R. (2017). Procesamiento de imágenes [Apuntes de cátedra]. Archivo Apuntes.pdf.

UADER. (2026). Computación Avanzada: Práctico Final Integrador [Consigna de cátedra]. Archivo CA-Practico Final-2026.pdf.

Datos de respaldo: resultados/matriz_completa/mediciones.csv, resumen.csv y validacion.txt. Los siete gráficos se generaron exclusivamente desde resumen.csv y mediciones.csv.